

ASSIGNMENT19.2

G.Mythili

2303A51283

Batch-05

TASK-01

Prompt : Convert the given Python file handling program into Java. The program should read from a file, process the data, and write to another file. Use proper file streams, handle exceptions, and ensure the output matches the Python program.

#Python program to read from a file and write to another file

try:

 # Open input file in read

 mode with open("input.txt", "r") as inf

 ile:

 data = inf.readlines()

 # Process data (convert each line to uppercase)

 processed_data = [line.upper() for line in data]

 # Open output file in write mode

 with open("output.txt", "w") as outfile:

 outfile.writelines(processed_data)

 print("File has been read and written successfully.")

except FileNotFoundError:

 print("Error: input.txt file not found.")

except Exception as e:

```
print("Anerroroccurred:",e)
```

Code:

```
[1] 0s %writefile FileHandler.java

import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.io.FileNotFoundException;

public class FileHandler {

    public static void main(String[] args) {
        String inputFile = "input.txt";
        String outputFile = "output.txt";

        try {
            BufferedReader reader = new BufferedReader(new FileReader(inputFile));
            BufferedWriter writer = new BufferedWriter(new FileWriter(outputFile))
        } {
            String line;
            while ((line = reader.readLine()) != null) {
                writer.write(line.toUpperCase());
                writer.newLine(); // Add a new line after each processed line
            }
            System.out.println("File has been read and written successfully.");

        } catch (FileNotFoundException e) {
            System.err.println("Error: " + inputFile + " file not found.");
        } catch (IOException e) {
            System.err.println("Error: " + inputFile + " file not found.");
        }
    }
}
```

Gemini

Python program to read from a file and write to another file try: # Open input file in read mode with open("input.txt", "r") as infile: data = infile.readlines() # Process data (convert each line to uppercase) processed_data = [line.upper() for line in data] # Open output file in write mode with open("output.txt", "w") as outfile: outfile.writelines(processed_data) print("File has been read and written successfully.") except FileNotFoundError: print("Error: input.txt file not found.") except Exception as e: print("An error occurred:", e) Convert this plan

What can I help you build?

+ Gemini 2.5 Flash ▶

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

```
[2] 0s %writefile input.txt
hello world
This is a test.
Python to Java conversion.

Writing input.txt

Now, you can compile and run the Java program. Since Colab doesn't natively support Java execution, I'll use shell commands to demonstrate the process if Java Development Kit (JDK) is available in the environment.

First, compile the Java source code:

[3] 3s # To compile the Java code (requires JDK installed in the environment)
!javac FileHandler.java

After successful compilation (which will create FileHandler.class if no errors), you can run the program:

[4] 0s # To run the compiled Java code
!java FileHandler

File has been read and written successfully.

Finally, you can check the content of output.txt to verify the program worked correctly:
```

Output:

```
# Display the content of the generated output.txt
!cat output.txt

HELLO WORLD
THIS IS A TEST.
PYTHON TO JAVA CONVERSION.
```

Explanation:

- Converts a Python file handling program into Java.

- Ensures reading from and writing to files works correctly.
- Uses proper Java file stream classes.
- Includes exception handling for errors.
- Maintains the same output as the Python program.

TASK-02

Prompt : Convert the given C++ sorting program (e.g., Bubble Sort or Selection Sort) into an equivalent Python program. Ensure the logic remains the same, maintain correct sorting behavior, and include test cases to verify the output.

```
#include <iostream>
```

```
using namespace std;
```

```
int main(){
```

```
    int arr[] = {5,2,9,1,5,6};
```

```
    int n=6;
```

```
    //Bubble Sort
```

```
    for(int i = 0; i < n-1; i++) {
```

```
        for(int j=0; j < n-i-1; j++){
```

```
            if(arr[j]>arr[j+1]){
```

```
                //swap
```

```
                int temp=arr[j];
```

```
                arr[j] = arr[j+1];
```

```
                arr[j+1] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```

//Printsortedarray

cout<< "Sorted array: ";

    for(inti=0;i<n;i++){

        cout<<arr[i] << " ";

    }

return0;
}

```

Code:

The screenshot shows a code editor with a Python program that implements the Bubble Sort algorithm. The program sorts an array of integers and prints the result. To the right of the code editor is a Gemini chat window. The chat history shows a user asking for a conversion of a C++ sorting program to Python, and Gemini responding with a Python implementation of Bubble Sort. The chat window also includes a prompt "What can I help you build?" and a "Gemini 2.5 Flash" model selector.

```

Python Equivalent of C++ Bubble Sort Program

This Python program implements the same Bubble Sort algorithm as your C++ code. It sorts an array of integers and prints the result.

def bubble_sort(arr):
    n = len(arr)
    # Traverse through all array elements
    for i in range(n - 1):
        # Last i elements are already in place
        for j in range(n - i - 1):
            # Traverse the array from 0 to n-i-1
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j] # Pythonic swap

# Test case
arr = [5, 2, 9, 1, 5, 6]
n = len(arr)

print("Original array:", arr)

bubble_sort(arr)

# Print sorted array
print("Sorted array:", arr)

```

Gemini chat history:

- User: #include <iostream> using namespace std; int main() { int arr[] = {5, 2, 9, 1, 5, 6}; int n = 6; // Bubble Sort for(int i = 0; i < n-1; i++) { for(int j = 0; j < n-i-1; j++) { if(arr[j] > arr[j+1]) { // swap int temp = arr[j]; arr[j] = arr[j+1]; arr[j+1] = temp; } } } // Print sorted array cout << "Sorted array: "; for(int i = 0; i < n; i++) { cout << arr[i] << " "; } return 0; }
- Gemini: Convert the given C++ sorting program (e.g., Bubble Sort or Selection Sort) into an equivalent Python program. Ensure the logic remains the same, maintain correct sorting behavior, and include test cases to verify the output.
- User: Certainly! I'll convert your C++ Bubble Sort
- User: What can I help you build?

Output:

```

Original array: [5, 2, 9, 1, 5, 6]
Sorted array: [1, 2, 5, 5, 6, 9]

```

Explanation:

- `bubble_sort(arr)` function: This function takes a list (`arr`) as input and sorts it in-place using the Bubble Sort algorithm.
 - `n=len(arr)`: Gets the number of elements in the list.
 - Outer loop (`for i in range(n - 1)`): This loop runs `n-1` times. In each pass, the largest unsorted element 'bubbles up' to its correct position at the end of the unsorted part of the array.
 - Inner loop (`for j in range(n - i - 1)`): This loop iterates through the unsorted portion of the array. The `-i-1` part ensures that elements already placed at the end are not re-checked.

- `if arr[j] > arr[j+1]:` Compares adjacent elements.
- `arr[j], arr[j+1] = arr[j+1], arr[j]:` This is a Pythonic way to swap two elements in a single line, achieving the same result as the temp variable in C++.
- **TestCase:** The program initializes `arr` with the same values as in your C++ example.
- **Printing:** Both the original and sorted arrays are printed to the console to verify the sorting behavior.

TASK-03

Prompt : Convert the given SQL query with GROUP BY and aggregate functions into an equivalent Pandas DataFrame operation using `groupby()` and aggregation methods. Ensure the output matches the SQL result

SELECT department, SUM(sales) AS total_sales FROM
employees

GROUP BY department;

Code:

Pandas Equivalent of SQL GROUP BY with Aggregate Functions

This Python program demonstrates how to convert an SQL GROUP BY query with an aggregate function (like SUM) into a Pandas DataFrame operation using `groupby()` and `agg()` methods.

```

import pandas as pd

# 1. Create a sample DataFrame that resembles the 'employees' table
data = {
    'employee_id': [1, 2, 3, 4, 5, 6],
    'department': ['HR', 'Sales', 'IT', 'Sales', 'HR', 'IT'],
    'sales': [100, 250, 150, 300, 120, 180]
}
employees_df = pd.DataFrame(data)

print("Original Employees DataFrame:")
display(employees_df)

# 2. Apply groupby() and aggregation to calculate total sales by department
total_sales_by_department = employees_df.groupby('department').agg(total_sales=('sales', 'sum')).reset_index()

print("Total Sales by Department (Pandas):")
display(total_sales_by_department)

```

BY and aggregate functions into an equivalent Pandas DataFrame operation using `groupby()` and aggregation methods. Ensure the output matches the SQL result

SELECT department, SUM(sales) AS total_sales FROM employees GROUP BY department;

◆ Certainly! I'll convert the SQL GROUP BY query to an equivalent Pandas operation. First, I'll create a sample DataFrame, and then I'll apply the `groupby()` and `agg()` methods to get the total sales by department.

The conversion of your SQL GROUP BY

What can I help you build?

+ Gemini 2.5 Flash ▶

Output:

Original Employees DataFrame:

	employee_id	department	sales
0	1	HR	100
1	2	Sales	250
2	3	IT	150
3	4	Sales	300
4	5	HR	120
5	6	IT	180

Total Sales by Department (Pandas):

	department	total_sales
0	HR	220
1	IT	330
2	Sales	550

Explanaton:

- `employees_df.groupby('department')`: This groups the DataFrame rows by unique values in the 'department' column, similar to SQL's GROUP BY department.
- `.agg(total_sales=('sales', 'sum'))`: This applies the aggregation. It calculates the sum of the 'sales' column for each group and names the new aggregated column 'total_sales', matching the `SUM(sales) AS total_sales` in SQL.
- `.reset_index()`: After `groupby()` and `agg()`, 'department' becomes the index. `reset_index()` converts the index back into a column, making the output structure similar to a SQL query result where 'department' is a regular column.

TASK-04

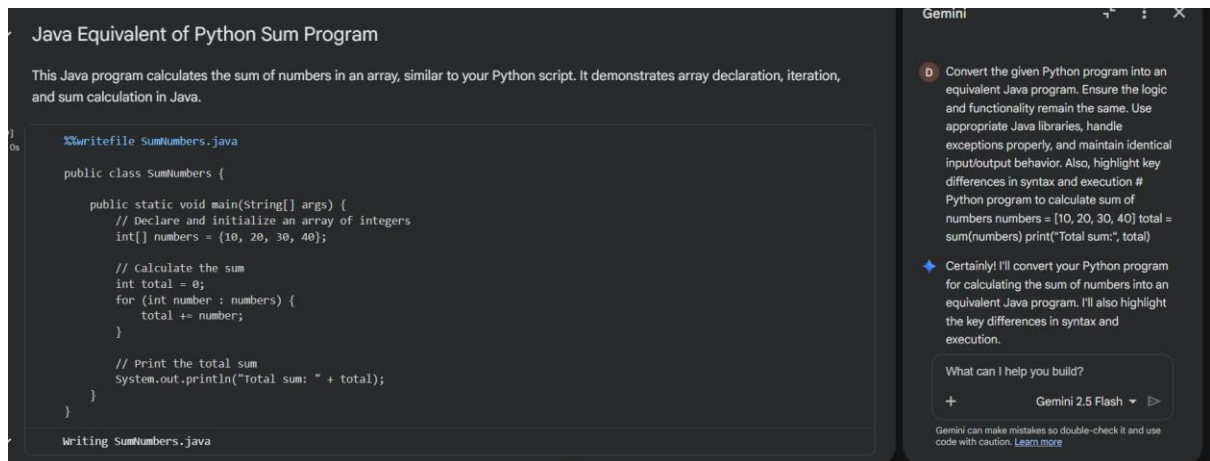
Prompt: Convert the given Python program into an equivalent Java program. Ensure the logic and functionality remain the same. Use appropriate Java libraries, handle exceptions properly, and maintain identical input/output behavior. Also, highlight key differences in syntax and execution

```
#Python program to calculate sum of numbers
numbers = [10, 20, 30, 40]

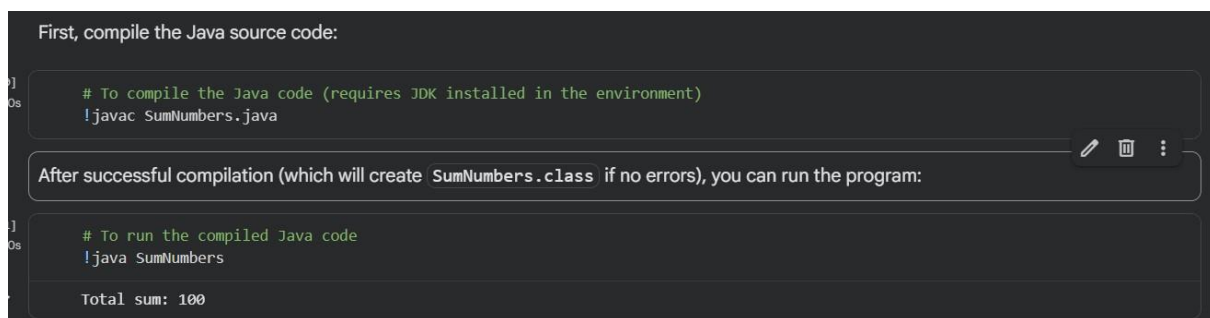
total = sum(numbers)

print("Total sum:", total)
```

Code :



Output:



Explanation:

- Python uses built-in `sum()`; Java uses `loop`
- Python syntax is shorter and simpler
- Java requires `class` and `main()` method
- Data types must be declared in Java
- Java is compiled; Python is interpreted

TASK-05

Prompt: Convert the given Python function that checks whether a number is even or odd into an equivalent C++ program. Ensure proper syntax, input handling, and output formatting, and verify that both programs produce the same results.

#Function to check even or odd

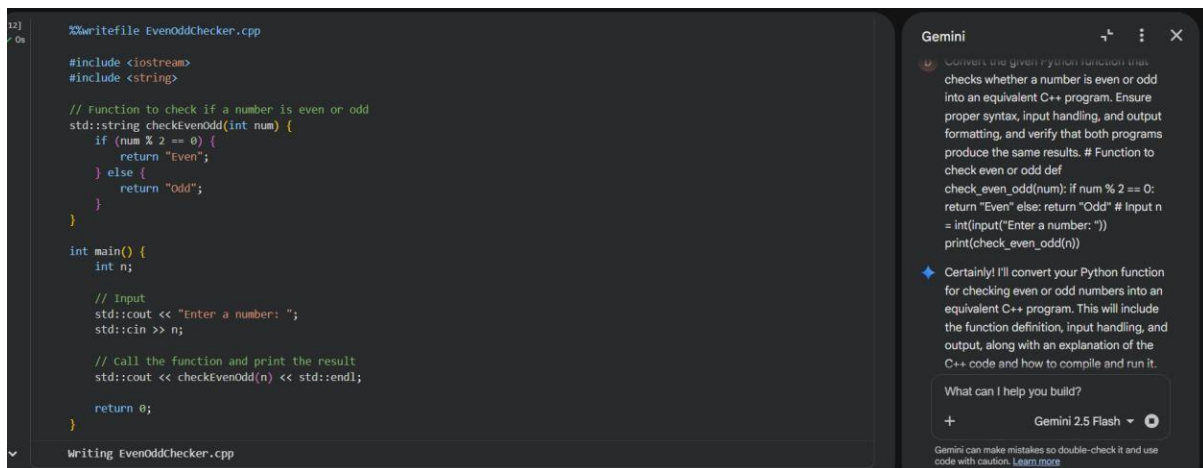
```
def check_even_odd(num):
    if num % 2 == 0:
        return "Even"
```

```
else:  
    return "Odd"
```

#Input

```
n=int(input("Enter a number:"))  
print(check_even_odd(n))
```

Code:



The screenshot shows a code editor with a C++ program named `EvenOddChecker.cpp`. The code includes headers for `<iostream>` and `<string>`, and defines a function `checkEvenOdd` that takes an integer and returns a string "Even" or "Odd" based on whether the number is divisible by 2. The `main` function prompts the user for input, reads it, calls `checkEvenOdd`, and prints the result. To the right, a Gemini chat window is open, displaying a conversation where the user asks for a C++ equivalent of a Python function, and Gemini provides the C++ code and an explanation of its logic.

```
12] // On %writefile EvenOddChecker.cpp  
  
#include <iostream>  
#include <string>  
  
// Function to check if a number is even or odd  
std::string checkEvenOdd(int num) {  
    if (num % 2 == 0) {  
        return "Even";  
    } else {  
        return "Odd";  
    }  
}  
  
int main() {  
    int n;  
  
    // Input  
    std::cout << "Enter a number: ";  
    std::cin >> n;  
  
    // Call the function and print the result  
    std::cout << checkEvenOdd(n) << std::endl;  
  
    return 0;  
}  
  
Writing EvenOddChecker.cpp
```

Gemini

Convert the given python function that checks whether a number is even or odd into an equivalent C++ program. Ensure proper syntax, input handling, and output formatting, and verify that both programs produce the same results. # Function to check even or odd def check_even_odd(num): if num % 2 == 0: return "Even" else: return "Odd" # Input n = int(input("Enter a number:")) print(check_even_odd(n))

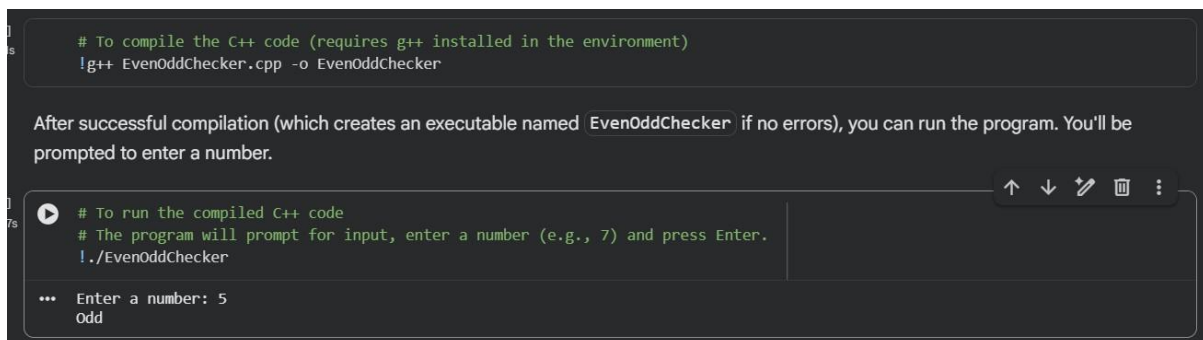
Certainly! I'll convert your Python function for checking even or odd numbers into an equivalent C++ program. This will include the function definition, input handling, and output, along with an explanation of the C++ code and how to compile and run it.

What can I help you build?

+ Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Output:



The screenshot shows a terminal window with the following content:

```
# To compile the C++ code (requires g++ installed in the environment)  
g++ EvenOddChecker.cpp -o EvenOddChecker
```

After successful compilation (which creates an executable named `EvenOddChecker` if no errors), you can run the program. You'll be prompted to enter a number.

```
# To run the compiled C++ code  
# The program will prompt for input, enter a number (e.g., 7) and press Enter.  
!./EvenOddChecker
```

Enter a number: 5
Odd

Explanation:

- The task converts a Python function into an equivalent C++ program.
- Both programs use the same logic ($\text{num} \% 2$) to check even or odd.
- Proper syntax and input/output handling are ensured in C++.
- The programs are tested with different inputs to verify correctness.
- Both versions produce the same output, confirming successful translation.